DATA-260 HOMEWORK 1
Sanjana Thummalapalli

PERSONAL CONFIGURATION

Student Name	Sanjana Thummalapalli
SID4	7801
PORT_BASE	8601
PREFIX	s7801
SEED	7801
VERIFY_SEED	267801
DOMAIN_ID	1
Assigned Domain	Clinical Trial Listings
Python Version	3.11
Local Model	qwen3:4b
Tagged Commit Hash	dc5790a443d753e8ee1e75b6e999b2d4e2106243
GitHub Link	https://github.com/sanjana-glitch-art/data260-7801

HARDWARE

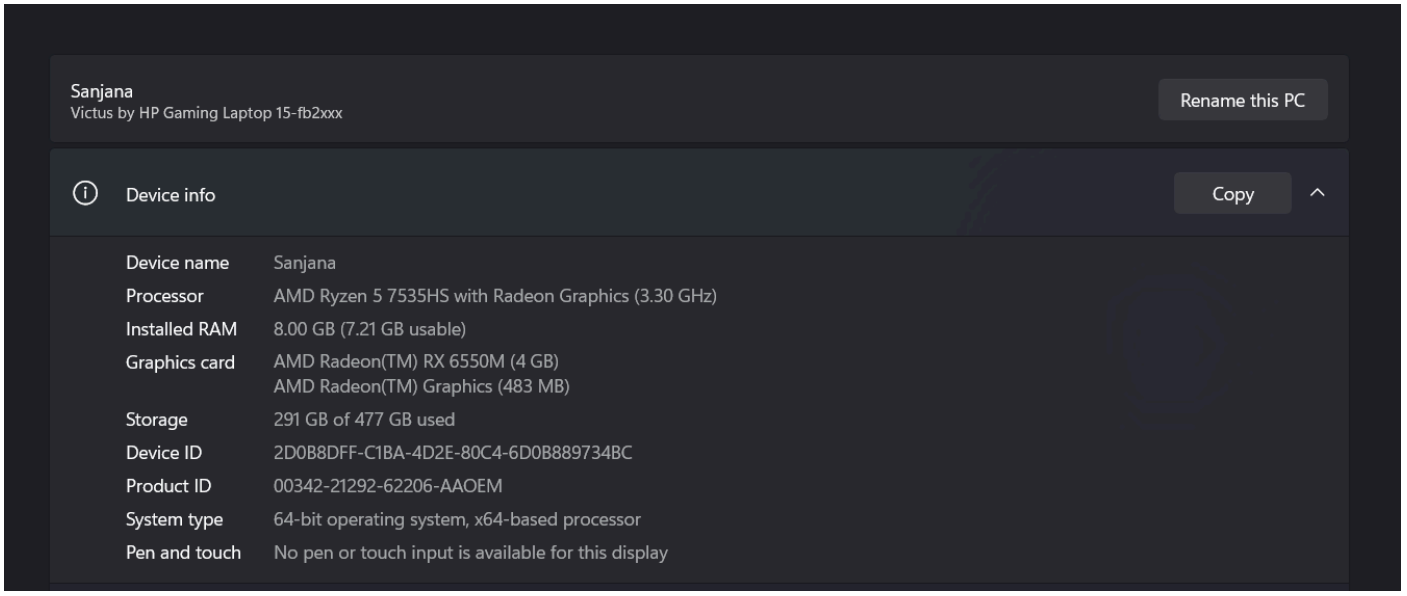

FIGURE 1: HARDWARE INFORMATION

MODEL SELECTION

I initially tested the requested qwen3:8b model. On my computer, one Planner-Reviewer pipeline run required approximately 519 seconds. My computer has 8 GB of RAM and a 4 GB AMD Radeon RX 6550M GPU. Because this runtime made the required 40-run experiment impractical, I used the smaller tool-capable qwen3:4b model.

PART 1 - HTML AND JAVASCRIPT

1. DOMAIN SCHEMA

Before implementing the HTML form, I defined the clinical-trial entity in DOMAIN_SCHEMA.md.
The primary field is trialTitle. The secondary field is sponsorName. The content field is trialDescription. The four category values are Phase I, Phase II, Phase III, and Phase IV.

Schema:

Field	Type	Required	Description
trialTitle	Text	Yes	Official or descriptive trial title
sponsorName	Text	Yes	Email address of the submitter
submitterEmail	Email	Yes	Email address of the submitter
trialDescription	Text ares	Yes	Description and purpose of the trial
trialPhase	Category	Yes	Phase I, II, III, or IV
termsAccepted	Boolean	Yes	Terms and conditions acceptance
submissionDate	Date and time	Generated	Successful submission timestamps

2. HTML FORM

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>HW1-Sanjana Thummalapalli</title>

    <style> ···
    </style>
</head>

<body>
    <h1>Clinical Trial Listing</h1>

    <form id="clinicalTrialForm">
        <label for="trialTitle">Trial title</label>
        <input
            type="text"
            id="trialTitle"
            name="trialTitle"
            placeholder="Enter the clinical trial title"
            required
            autofocus
        >

        <label for="sponsorName">Sponsor name</label>
        <input
            type="text"
            id="sponsorName"
            name="sponsorName"
            placeholder="Enter the sponsoring organization"
            required
        >

        <label for="submitterEmail">Submitter email</label>
        <input
            type="email"
            id="submitterEmail"
            name="submitterEmail"
            placeholder="Enter the submitter's email address"
            required
        >

        <label for="trialDescription">Trial description</label>
        <textarea
            id="trialDescription"
            name="trialDescription"
            placeholder="Describe the clinical trial, its purpose, and its participants"
```

FIGURE 2: RELEVANT HTML FORM CODE

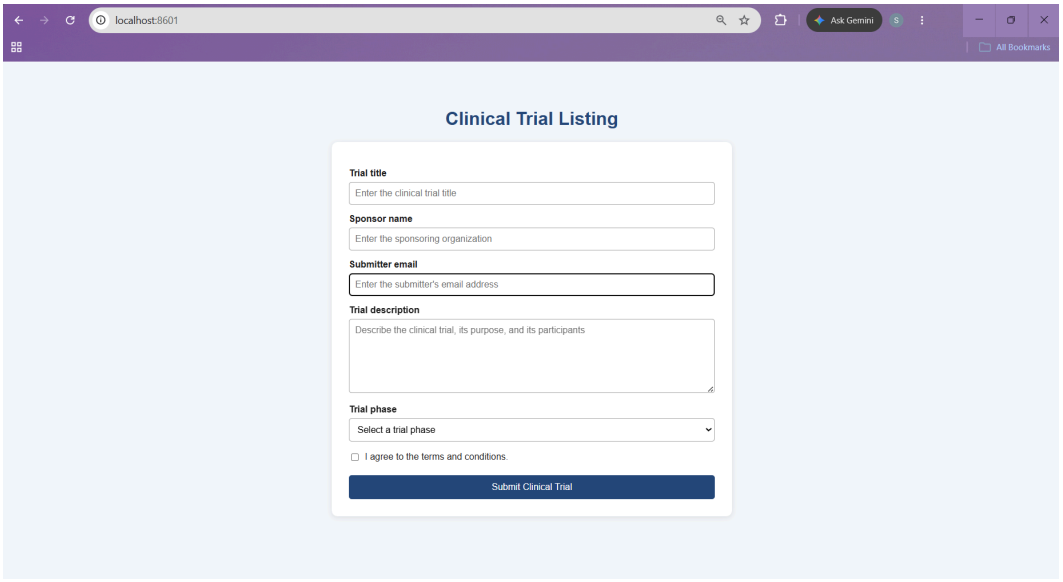

FIGURE 3: FORM RUNNING AT http://localhost:8601

3. JAVASCRIPT VALIDATION

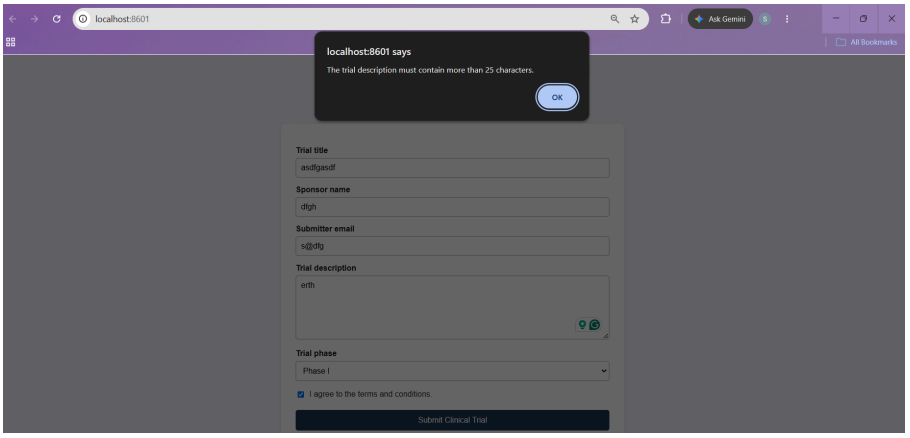

FIGURE 4: DESCRIPTION VALIDATION ALERT

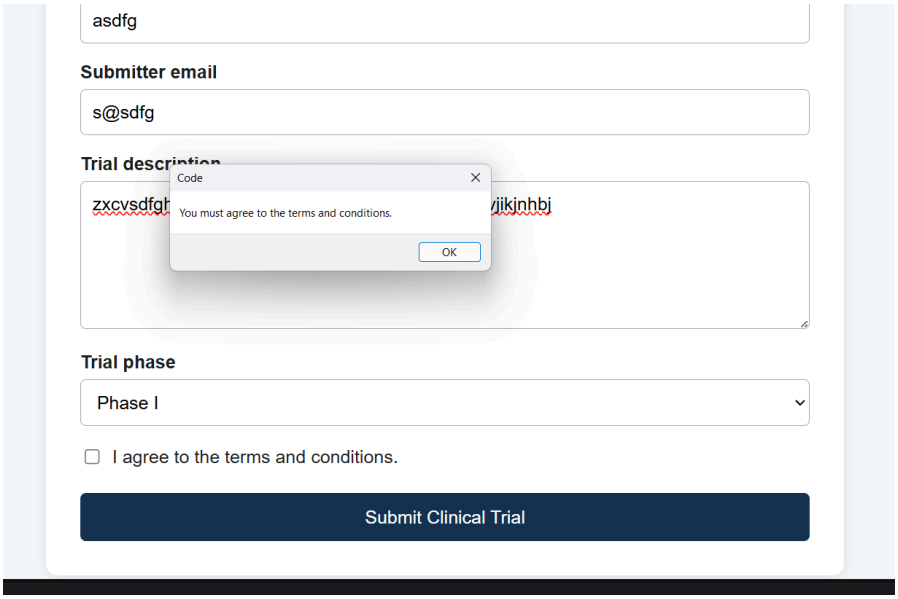

FIGURE 5: TERMS VALIDATION ALERT

4. JSON, DESTRUCTURING, SPREAD OPERATOR, AND CLOSURE

After validation succeeds, the form data is converted to a JSON string and logged. The JSON string is parsed back into an object. Object destructuring extracts trialTitle and submitterEmail. The spread operator adds submissionDate. A closure tracks successful submissions.

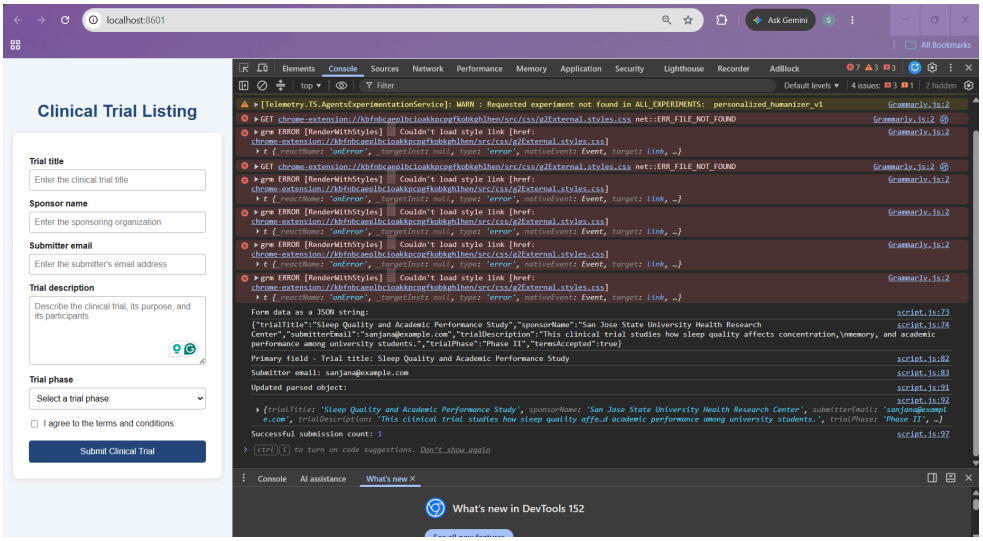

FIGURE 6: SUCCESSFUL FORM AND CONSOLE OUTPUT

5. LOCAL DOCKER DEPLOYMENT

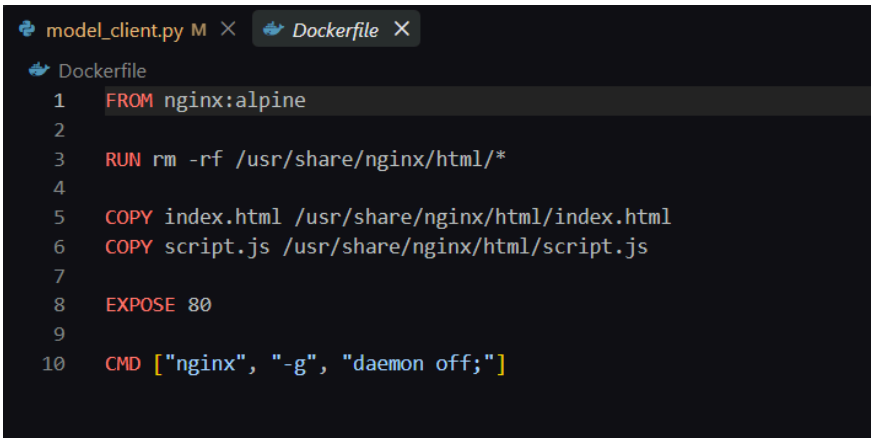

FIGURE 7: dockerfile

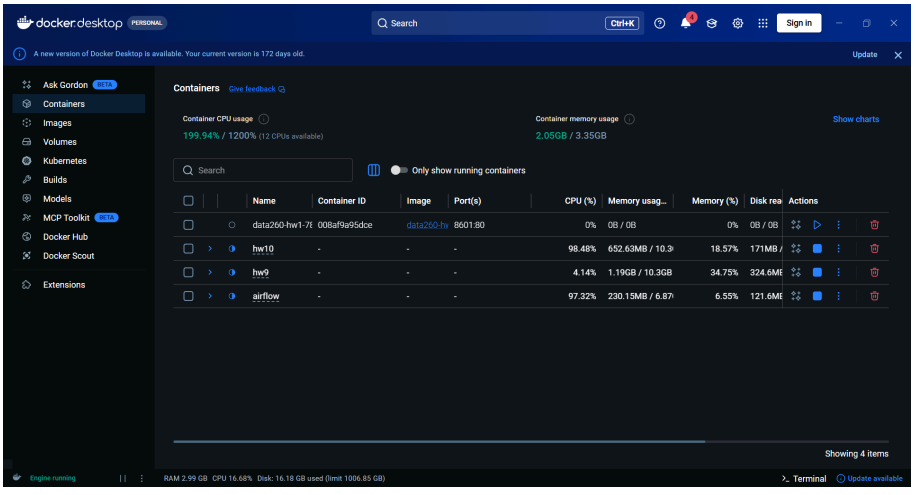

FIGURE 8: docker-ps

6. AWS ECS DEPLOYMENT

My student AWS credits were exhausted. The TA approved attempting the deployment using a classmate's account, but AWS account issues prevented the deployment from being completed before this report was prepared. The local Docker deployment was completed successfully. I will replace this statement with the completed ECS evidence if access becomes available before submission.

PART 2 - AGENTIC AI

1. DESIGN

I implemented two model agents and one non-agent finalization step:

- Planner: proposes exactly three topical tags and a summary.
- Reviewer: checks topicality, uniqueness, and the 25-word summary limit.
- Finalizer: deterministically guarantees exactly three tags and a summary of at most 25 words.

All model calls pass through the reusable adapter in src/model_client.py. The agent logic does not contain fixed clinical-trial tags. Tags and summaries are derived from the supplied title and content.

2. COMMAND

python agents_demo.py --model qwen3:4b --title "Sleep Quality and Academic Performance Study" --content "This clinical trial studies how sleep duration and sleep quality affect concentration, memory, and academic performance among university students."

3. PLANNER AND REVIEWER CODE

```
    result = model.complete(
        messages,
        response_format=PLANNER_SCHEMA
    )

    return extract_json(result.content)
```

```
    result = model.complete(
        messages,
        response_format=REVIEWER_SCHEMA
    )

    reviewer_output = extract_json(
        result.content
    )
```

FIGURE 9: planner and reviewer code

The Reviewer's changed field is validated by directly comparing the Reviewer tags and summary with the Planner output. This prevents the model from claiming that it changed something when the visible values are identical.

```
(.venv) PS C:\Drive E\projects\data260-7801> python agents_demo.py --model qwen3:4b --title "Sleep Quality and Academic Performance Study" --content "This clinica
l trial studies how sleep duration and sleep quality affect concentration, memory, and academic performance among university students."

--- Planner Output ---
{
  "tags": [
    "sleep_duration",
    "academic_performance",
    "student_concentration"
  ],
  "summary": "Clinical trial examining sleep impact on university students' concentration and academic outcomes."
}
Planner latency: 14549.11 ms

--- Reviewer Output ---
{
  "tags": [
    "sleep_duration",
    "academic_performance",
    "student_concentration"
  ],
  "summary": "Clinical trial examining sleep impact on university students' concentration and academic outcomes.",
  "changed": false,
  "changes": []
}
Reviewer latency: 2729.44 ms
```

FIGURE 10 : PLANNER AND REVIEWER OUTPUT

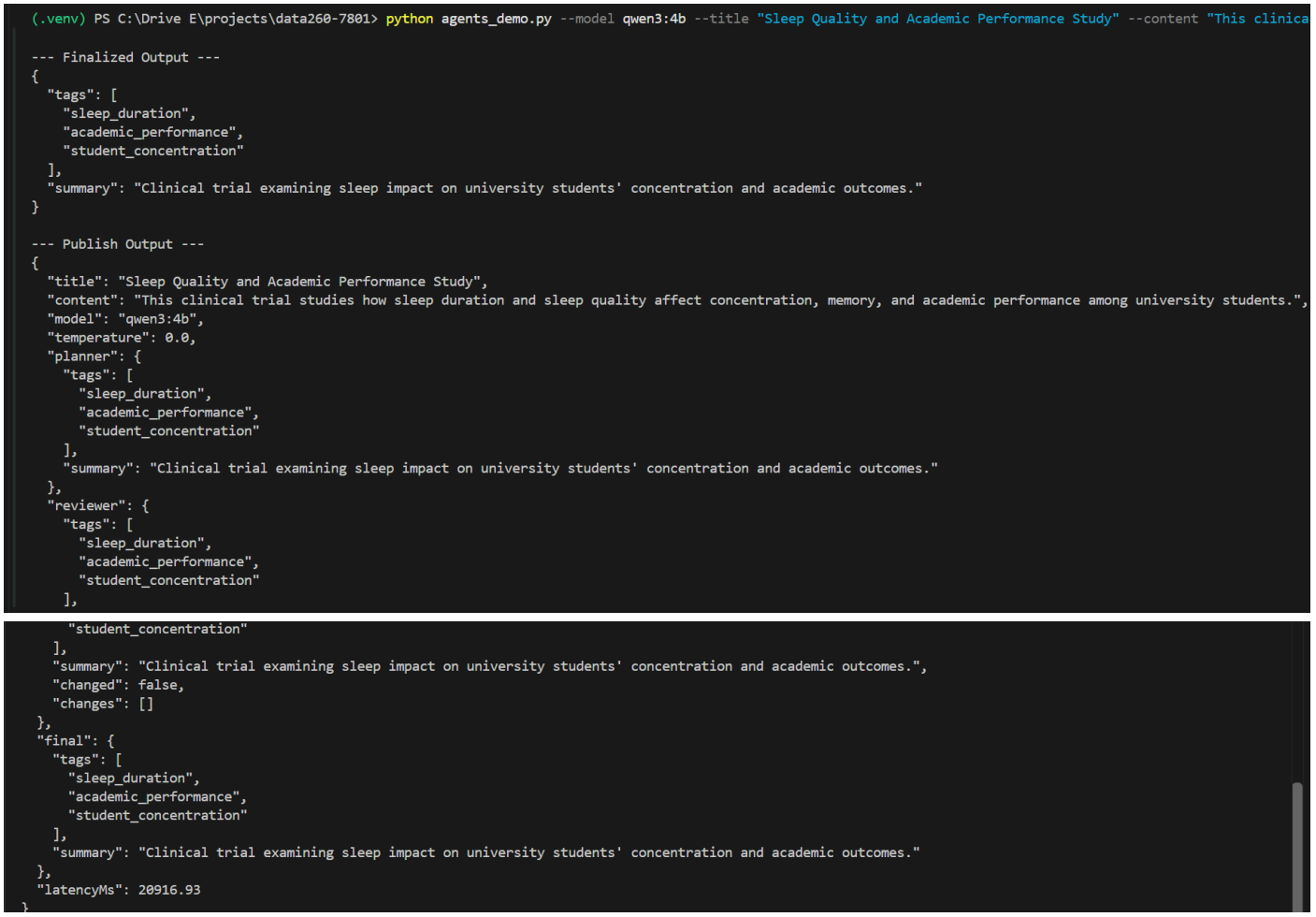

FIGURE 11:FINALIZED AND PUBLISHED OUTPUT

Q1. What three final tags were obtained?

- sleep_duration
- academic_performance
- student_concentration

Q2. What was the final summary?
Clinical trial examining sleep impact on university students' concentration and academic outcomes.

Q3. Did the Reviewer change anything?
No. The Reviewer determined that the Planner's three tags were topical and distinct and that its summary was already under the 25-word limit.

5. STEP EXPLANATION

The Planner receives the fixed title and content and creates a structured proposal. A JSON schema forces the proposal to contain exactly three tags and a summary.

The Reviewer receives the original title and content along with the Planner proposal. It checks whether the tags are topical and distinct and whether the summary follows the word limit.

The finalization step normalizes the tags, removes duplicates, fills missing tags using phrases derived from the input, and truncates the summary if it exceeds 25 words. It is a Python validation step rather than a third model agent.

The final Publish output contains the title, content, model name, temperature, Planner output, Reviewer output, final tags and summary, and measured latency.

PART 3 - MEASURING NONDETERMINISM

1. FIXED INPUT

The unchanged input was saved to:
reports/hw01/cases/nondeterminism_input.json

{
"title": "Sleep Quality and Academic Performance Study",

"content": "This clinical trial studies how sleep duration and sleep quality affect concentration, memory, and academic performance among university students."
}

2. EXPERIMENT

I ran the complete Planner-Reviewer-Finalizer pipeline:

- 20 times at temperature 0.7
- 20 times at temperature 0.0
- 40 successful runs in total

Each result contains its temperature, run number, timestamp, three tags, summary, Planner latency, Reviewer latency, total latency, and Reviewer status.

Raw results were saved as:

- reports/hw01/raw/nondeterminism_results.json
- reports/hw01/raw/nondeterminism_results.csv
- reports/hw01/raw/nondeterminism_metrics.json

3. TAG RESULTS

Metric	Temperature 0.7	Temperature 0.0
Distinct tag sets	11	1
Tags in all 20 runs	None	Academic_performance, concentration_effects, sleep_duration
Tags appearing in exactly one run	academic_concentration; concentration_impact; concentration_metrics; concentration_outcomes; student_cognitive_function; student_concentration_studies; university_sleep_studies	None

4. LATENCY RESULTS

Metric	Temperature 0.7	Temperature 0.0
p50	5526.03 ms	5105.43 ms
p95	7654.37 ms	5529.57 ms
p99	18532.39 ms	7052.89 ms

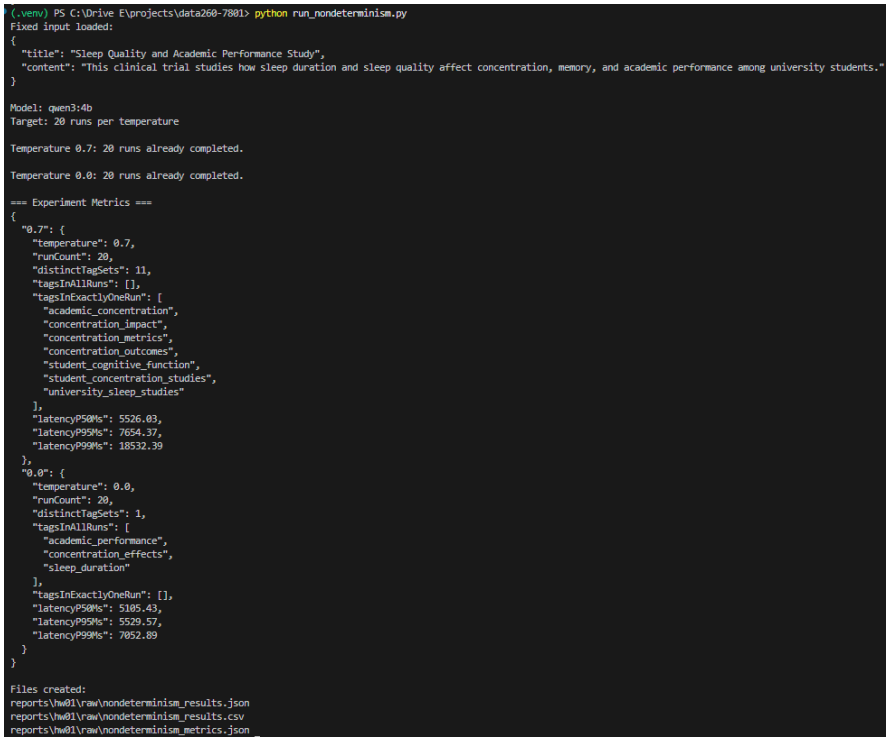

FIGURE12: NONDETERMINISM METRICS

5. INTERPRETATION

At temperature 0.7, two users sending the same title and content may receive different but related tag sets. For example, one user might receive sleep_quality, academic_performance, and university_students, while another might receive sleep_duration, academic_performance, and student_concentration. At temperature 0.0, the same input produced the same tag set in all 20 runs. Temperature 0.0 therefore produced more repeatable results.

Variation is acceptable for brainstorming, discovery, and generating alternative topical tags because multiple descriptions may be equally useful. Variation is unacceptable when the output determines clinical-trial eligibility, participant safety, or another consequential decision. Such decisions require validated rules, consistency, and human review.

PART 4 - MODEL CLIENT AND TOKEN ACCOUNTING

1. REUSABLE ADAPTER

The reusable adapter is implemented in src/model_client.py. Its stable interface is:

```
def complete(
    self,
    messages,
    tools=None,
    response_format=None
):
```

It sends requests to the local Ollama server and returns:

- Response content
- Input tokens
- Output tokens
- Total tokens
- Raw response metadata

All project model calls use this adapter.

2. AGENT.md

AGENT.md instructs the model to produce strict bullet-only code reviews. The interactive client also validates that every nonempty displayed response line begins with "- ".
All five responses passed this verification.

3. TOKEN RESULTS

Turn	Input Tokens	Output Tokens	Total Tokens	Cumulative Input	Cumulative Output	Serialized History
1	134	46	180	134	46	249

2	191	33	224	325	79	469
3	241	61	302	566	140	881
4	313	73	386	879	213	1298
5	403	53	456	1282	266	1667

4. /stats AFTER TURN 3

Turn count: 3
Cumulative input tokens: 566
Cumulative output tokens: 140
Cumulative total tokens: 706
Serialized conversation-history length: 881 characters

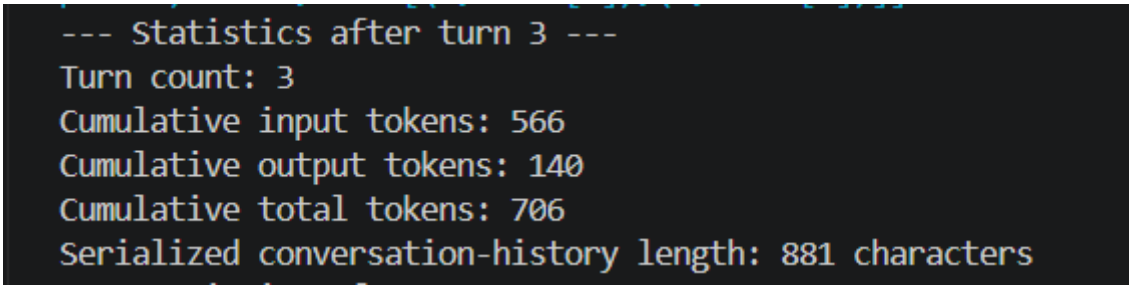

FIGURE 13: /stats AFTER TURN 3

5. /stats AFTER TURN 5

Turn count: 5
Cumulative input tokens: 1,282
Cumulative output tokens: 266
Cumulative total tokens: 1,548
Serialized conversation-history length: 1,667 characters

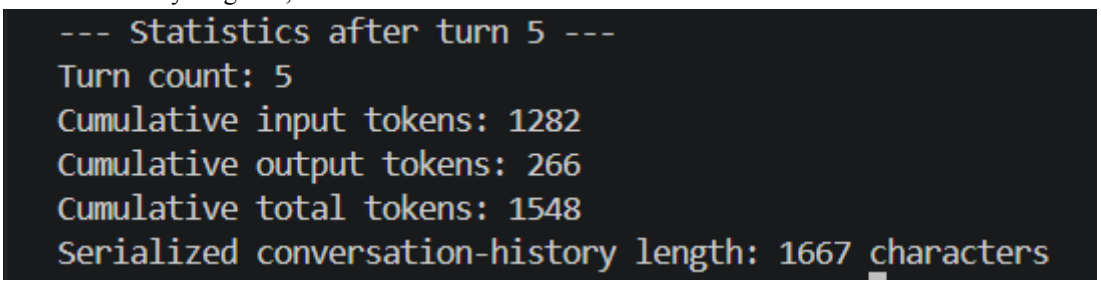

FIGURE 14: /stats AFTER TURN 5

6. REQUIRED QUESTIONS

Why is prior conversation context resent with every turn?
A model request is normally stateless. The application must resend earlier messages so the model can understand references to previous questions and answers and continue the conversation coherently.

How is a system prompt different from a user message?
A system prompt defines the model's overall role, behavioral constraints, and response format. A user message contains the specific task or question. System instructions have higher priority and remain applicable throughout the conversation.

Why do input tokens grow over a conversation?
Each request includes the system prompt, earlier user messages, earlier assistant responses, and the newest user message. As that serialized history becomes longer, the number of input tokens grows.

What eventually limits that growth?
The model's context-window limit eventually restricts the number of tokens that can be included in one request. An application must then remove old messages, summarize them, or compress the conversation history.

VERIFICATION
I created verify_hw1.py to verify:

- Required files
- Python version
- HTML requirements

- JavaScript requirements
- Model-adapter use
- Forty nondeterminism results
- Five-turn token accounting
- Bullet-only compliance

The verification output reported passed: true.

FIGURE 15: VERIFICATION OUTPUT

REPRODUCIBLE RUN INSTRUCTIONS
Python environment:
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
ollama pull qwen3:4b

Local Docker application:
docker build -t data260-hw1:latest .
docker run --name data260-hw1-7801 -d -p 8601:80 data260-hw1:latest

Agent pipeline:
python agents_demo.py --model qwen3:4b --title "Sleep Quality and Academic Performance Study" --content "This clinical trial studies how sleep duration and sleep quality affect concentration, memory, and academic performance among university students."

Nondeterminism experiment:
python run_nondeterminism.py

Interactive client:
python hw1_client.py

Verification:
python verify_hw1.py